PROJECT REPORT

Saywise

Local-first English speaking practice with transparent speech evidence

Project type Flask web application | **Primary stack** Python, Flask, SQLite, Faster-Whisper

Saywise helps learners practise spoken English using local transcription, word-level comparison, and measurable timing features. Its interface intentionally separates observable evidence from unvalidated pronunciation judgments.

Prepared from the application source, interface, tests, and project documentation.

November 2026

1. Executive summary

Saywise is a local-first practice application for English speech. Learners select a built-in prompt or enter their own text, record a short utterance in the browser, and review a transcript, lexical differences, timing summaries, and practice suggestions. User accounts and practice history are handled by a Flask API backed by SQLite.

The application is careful about what its outputs mean. Faster-Whisper transcribes speech; a deterministic edit alignment compares that transcript with the prompt; and an optional wav2vec2 model recognizes phone labels. Transcript edits and phone-sequence differences are not proof of mispronunciation. The application therefore withholds a numeric pronunciation score until a human-labeled evaluation supports one.

Project goals

- Offer a usable, level-based spoken-English practice workflow.
- Show substitutions, deletions, and insertions without conflating ASR errors with pronunciation errors.
- Summarize speech duration, estimated active-speech time, rate, and pauses without inventing a fluency grade.
- Protect learner history with authentication, per-user ownership, CSRF checks, and opt-in recording retention.
- Provide a credible path for validating future pronunciation assessment against labeled data.

2. Product and user workflow

A learner registers or signs in, chooses a practice level, selects one of the seeded sentences or enters custom text, records and previews speech, and submits the clip for analysis. Results appear in the session review and can be revisited through practice history. The interface also includes profile controls, a daily challenge, progress summaries, and an administrator aggregate-count view.

Stage	What the application does
Prompt	Presents a level-matched sentence, or accepts learner-supplied text.
Capture	Uses browser recording; accepts audio uploads within configured limits (15 MB and 60 seconds).
Analyze	Decodes audio, transcribes locally, aligns recognized words, estimates timing, and attempts optional phone recognition.
Review	Displays transcript evidence, practice guidance, and status/limitations for optional analysis.
History	Stores attempt text and analysis metadata; retained audio is optional and user-deletable.

3. System design and technology

Layer	Implementation
Web/API	Flask application factory, JSON endpoints, browser UI in HTML/CSS/JavaScript.
Speech-to-text	Faster-Whisper (`small.en` by default), with word timestamps; model runs locally after weights are available.
Audio processing	PyAV decoding and resampling to mono 16 kHz; NumPy-based short-window RMS timing heuristic.
Optional phones	wav2vec2 phone-label recognizer plus eSpeak NG/Phonemizer reference phones; explicitly experimental.
Persistence	SQLite accounts, prompts, attempts, transcript/analysis JSON, and optional recording paths.
PDF export	ReportLab-powered per-attempt PDF download is implemented in the app.

Request path

Browser ? authenticated Flask API ? input validation/audio decode ? ASR + timing + optional phone recognition ? deterministic comparison and feedback ? SQLite attempt history ? browser results.

4. Evidence and algorithms

Word alignment tokenizes case-insensitive alphanumeric words and applies minimum-edit dynamic programming. It backtraces the match, substitution, deletion, and insertion sequence, and computes word error rate (WER) as edit count divided by reference-word count. This is a lexical comparison: an edit may come from recognition uncertainty, recording conditions, or a spoken variation.

Signal	Method	Interpretation and boundary
Transcript	Faster-Whisper English ASR	Text hypothesis, not a pronunciation verdict.
Word edits	Levenshtein alignment	Identifies lexical differences; cannot establish their cause.
Timing	RMS activity over 20 ms frames; inactive gaps merged	Estimated active speech and pauses; sensitive to noise and quiet speech.
Phone sequence	Optional wav2vec2 CTC labels vs. eSpeak reference	Experimental review cue only; phone inventory/stress may differ.
Pronunciation score	Not calculated	No scalar score or validated correctness confidence is exposed.

Feedback is rule-based and tied to available evidence. Transcript edits prompt a careful repetition; a detected pause can prompt phrase-level practice; repeated mismatch suggestions are explicitly described as possible ASR uncertainty. The system does not report fabricated phone substitutions or a composite grade.

5. Data, privacy, and security

The SQLite schema contains users, seeded sentences, and attempts. Passwords are stored as Werkzeug password hashes. Mutating API endpoints require a session CSRF token; analysis and history routes require sign-in, and attempt queries are scoped to the current user. Upload size and duration are capped and audio is decoded before inference.

Audio is deleted after analysis by default. A learner may opt to retain it, replay retained audio, or delete an attempt, all practice history, or the account. Transcripts and analysis metadata remain in history until the learner deletes them. The Flask development server is intended for local use; a public deployment requires HTTPS, a persistent secret, a production WSGI server, and operational/privacy controls.

6. Verification and current results

The repository includes automated tests for word alignment, empty-input behavior, WER, timing summaries, phoneme-error-rate and detection metrics, registration/password hashing, CSRF protection, authenticated routes, custom prompts, and database migration. These tests verify application behavior and deterministic utilities; they do not measure ASR quality, pronunciation accuracy, accent robustness, or learner outcomes.

No model benchmark or pronunciation-scoring result is claimed in this project. Evaluation functions are utilities, not evidence that the recognizer performs at a particular accuracy. The report draft recommends speaker-disjoint validation and held-out testing against human annotations.

Recommended research measures

- ASR WER and phone error rate (PER), after documenting token/phone normalization.
- Error-detection precision, recall, and F1 against human-labeled error categories.

- For continuous human ratings: MAE and correlation/agreement, alongside inter-rater variability.
- Speaker-level splits, uncertainty intervals, class balance, and supported accent/L1 subgroup results.

7. Limitations and responsible interpretation

The phone recognizer was trained to recognize phonetic labels, not to assess learner pronunciation quality. Reference phones and predicted phones can differ because of stress or inventory conventions. ASR mismatch cannot distinguish pronunciation from recognition errors. RMS activity detection is a simple baseline. The project has no trained calibration model, custom fine-tuning, or reported performance results; optional phone analysis also needs extra dependencies and a first-run model download.

The web interface loads Google Fonts and Chart.js from CDNs. Speech inference is local after model weights have been downloaded and cached, but first-run weights may be fetched from their hosting service. The optional phoneme stack depends on PyTorch, Transformers, Phonemizer, and eSpeak NG.

8. Roadmap and conclusion

- Evaluate baseline phone/timing features on a licensed, speaker-disjoint corpus such as SpeechOcean762; tune only on validation speakers.
- Use L2-ARCTIC or a consented, human-annotated learner sample for cross-accent checks, subject to license and ethics review.
- Add calibration and an uncertainty/abstention path only if held-out performance justifies a score.
- Improve voice activity detection, accessibility, offline frontend assets, deployment hardening, and persistence options.

Saywise delivers a complete local practice workflow with clear transcript evidence, useful timing summaries, account/history management, and optional experimental phone comparison. Its most important design decision is to avoid presenting these proxies as a validated pronunciation score.

Selected references

[1] Zhang et al., ?speechocean762: An Open-Source Non-native English Speech Corpus for Pronunciation Assessment,? Interspeech 2021. [2] Zhao et al., ?L2-ARCTIC: A Non-native English Speech Corpus,? Interspeech 2018. [3] Xu, Baevski & Auli, ?Simple and Effective Zero-shot Cross-lingual Phoneme Recognition,? 2021. [4] SYSTRAN, Faster-Whisper project. [5] Meta AI, wav2vec2-lv-60-espeak-cv-ft model card.